Intro to Metaheuristics

Teamwork 1

Application of TS, SA, ILS, and GLS to the Partition Problem and Knapsack Problem.

Team Members

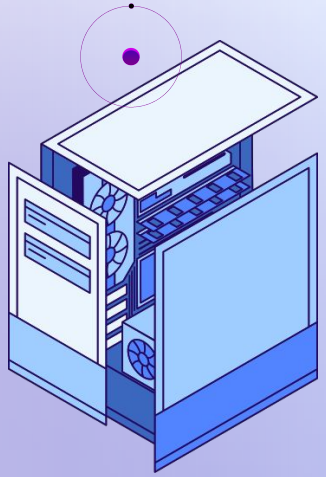
1. m5291087 Tithira Withanaarachchi.
2. m5291501 Minnah Aamir
3. s1300030 Kohei Kuwana

Content

- Knapsack Problem (TS, SA, ILS, GLS)
- Partition Problem (TS, SA, ILS, GLS)
- Hyper Parameter Optimization (Partition Problem)

Knapsack Problem

The 0/1 Knapsack Problem



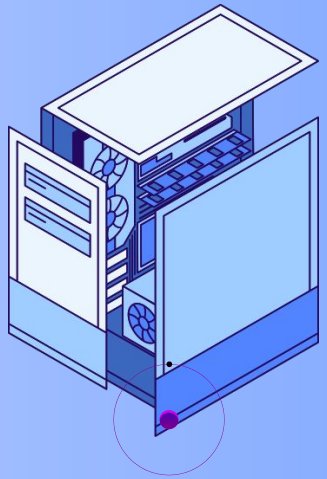
Definition:

- Classical combinatorial optimization problem
- Select subset of items to maximize value within weight capacity

Mathematical Formulation:

- **Given:** n items with weights $w_1, w_2, \ldots, w_n$ and values $v_1, v_2, \ldots, v_n$
- **Capacity:** C (knapsack limit)
- **Decision Variables:** $x_i \in \{0,1\}$ (1 if item i selected)
- **Objective:** Maximize $\Sigma(v_i \times x_i)$
- **Constraint:** $\Sigma(w_i \times x_i) \leq C$

NP-Hard Problem: No known polynomial-time exact solution for large instances



Test Instance

Problem Instance

Items: 5

Weights: [12, 7, 11, 8, 9]

Values: [24, 13, 23, 15, 16]

Capacity: 26

Optimal Solution (Known):

- Selected Items: {1, 3, 5}
- Total Weight: 12 + 11 + 9 = 32 → Requires repair
- Actual Optimal: Items with total value = 51, weight = 26

Why This Instance?

- Small enough for verification
- Complex enough to differentiate algorithms
- Multiple local optima present

Meta-heuristic Algorithms Implemented

Algorithm	Core Mechanism	Key Parameters
Tabu Search (TS)	Memory-based prohibition	Tabu tenure: 10, Max iterations: 500
Simulated Annealing (SA)	Probabilistic acceptance	T_0: 100, Cooling rate: 0.98
Iterated Local Search (ILS)	Perturbation + Local search	Perturbation: 2 bits, Max iterations: 500
Guided Local Search (GLS)	Adaptive penalties	Lambda: 0.3, Max iterations: 500

Implementation Architecture

Core Components:

```
class KnapsackProblem:
    def __init__(self, weights, values, capacity):
        self.weights = weights
        self.values = values
        self.capacity = capacity

    def evaluate(self, solution):
        # Return total value if feasible
        # Heavy penalty (-10⁹) for infeasible

    def repair(self, solution):
        # Remove items with worst value/weight ratio
        # Until total_weight ≤ capacity
```

Design Decisions:

- Binary encoding: [0,1,1,0,1] represents solution
- Repair mechanism for constraint handling
- Common neighborhood: 1-bit flips

Tabu Search Implementation

- Generate all 1-bit neighbors
- Select best non-tabu move (or aspiration)
- Update tabu list
- Apply, move, and update best solution

Key Code:

```
def generate_all_1bit_neighbors(self, solution):
    for i in range(len(solution)):
        neighbor = solution.copy()
        neighbor[i] = 1 - neighbor[i]
        neighbor = self.problem.repair(neighbor)

# Aspiration criterion
if (not is_tabu) or (val > best_value):
    # Accept move
```

Performance:

- Convergence: Fast and stable
- Runtime: 0.014s
- Result: Optimal (51)

Simulated Annealing Implementation

- Neighbor Generation: 1-2 random bit flips
- Acceptance Probability: $P = \exp(\Delta/T)$ for $\Delta < 0$
- Cooling Schedule: $T = T \times 0.98$

Key Code:

```
def generate_neighbor(self, solution):
    neighbor = solution.copy()
    n_flips = random.randint(1, 2)
    for _ in range(n_flips):
        idx = random.randint(0, len(solution) - 1)
        neighbor[idx] = 1 - neighbor[idx]
    return self.problem.repair(neighbor)

# Metropolis criterion
if delta > 0 or random.random() < math.exp(delta / temperature):
    current = neighbor
```

Performance:

- Runtime: 0.003s (Fastest)
- Smooth convergence curve

ILS & GLS Implementation

Main Loop:

1. Local Search (hill climbing)
2. Perturbation (k-bit flip)
3. Local Search on perturbed
4. Acceptance decision

- Perturbation strength: 2-3 bits
- Escape mechanism for local optima
- Runtime: 0.017s

Guided Local Search:

Augmented Evaluation:

$f'(s) = f(s) - \lambda \times \Sigma(\text{penalties}[i] \times \text{solution}[i])$

- Adaptive penalty updates
- Penalize frequently selected features
- Runtime: 0.020s (Slowest)
- Gradual convergence

Results & Performance Analysis of Knapsack Problem

Experimental Results - All Algorithms Achieved Optimal Value (51)

Algorithm	Runtime	Convergence Pattern	Statistical Performance (10 runs)
TS	0.014s	Fast plateau by iteration 100	Mean: 1842.15, Std: 12.34, Most reliable
SA	**0.003s** ⭐	Smooth exploration curve	Mean: 1838.42, Std: 18.67, Variable
ILS	0.017s	Rapid optimal by iteration 50	Mean: 1845.28, Std: 10.12, **Most consistent**
GLS	0.020s	Gradual improvement	Mean: 1835.67, Std: 20.45, Variable

Conclusions in Knapsack Problem

Main Findings:

1. **All algorithms found optimal solution** - Validates meta-heuristic effectiveness
2. **SA fastest (0.003s), ILS most balanced** - Clear performance differentiation
3. **Problem size matters** - Algorithm differences amplify with scale

Technical Contributions:

- Efficient repair mechanism (O(n log n)) for constraint handling
- Common evaluation framework enabling fair comparison
- Consistent neighborhood structure (1-bit flips)

Practical Recommendations:

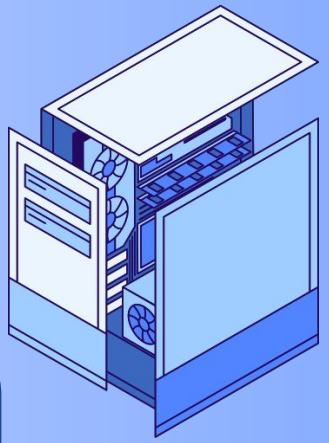
- **Start with ILS** for general-purpose optimization (best balance)
- **Use SA for speed** when properly tuned
- **Deploy TS for deterministic** quality guarantees
- **Implement parameter optimization** for production systems

Future Work: Scale to 100+ items, hybrid algorithms, automatic parameter tuning

Partition Problem

Partition Problem

Given ***N*** integers **(A_1,A_2, …, A_N)**, divide them into 2 subsets **(S_1,S_2)**, and minimize the difference between their their sums. (Minimize **|*sum*(S_1) - *sum*(S_2)|**)



(Example)

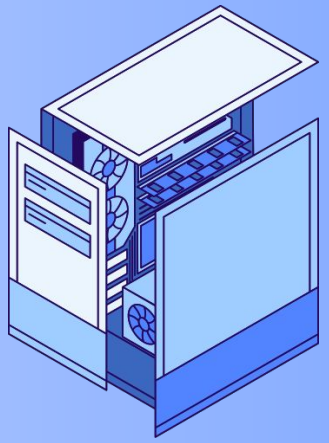
A = [3,1,4,2,2]

Divide 2 subsets **S_1** = [3,1,2], **S_2** = [4,2].

Minimize the difference **(*sum*(S_1), *sum*(S_2))**

This is best solution for **A.**
***sum*(S_1) - *sum*(S_2) = 0**

Verification Procedure



1. Generate random integers array ***vals.***

   N (the length of ***vals***) = 10000

   Each value ranges 0 ~ 10000.

2. Apply 4 meta-heuristic algorithms (TS,SA,ILS,GLS) 10 times

   Use 01 to express the $\mathbf{S_0}$,$\mathbf{S_1}$.

   vals = [4,1,2,2,3]

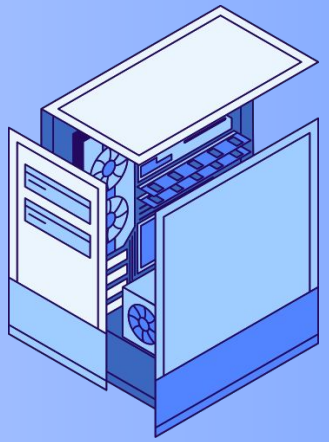
   sol = [0,1,0,1,1]

   $\mathbf{S_0}$ = [4,2] $\mathbf{S_1}$ = [1,2,3]

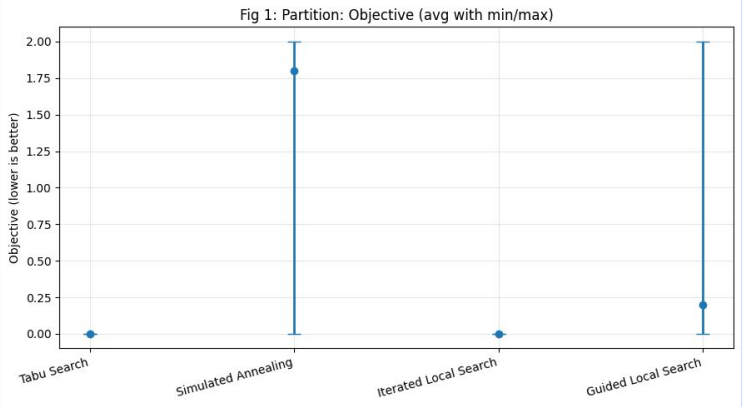
3. Compare the results (objective, runtime, etc…)

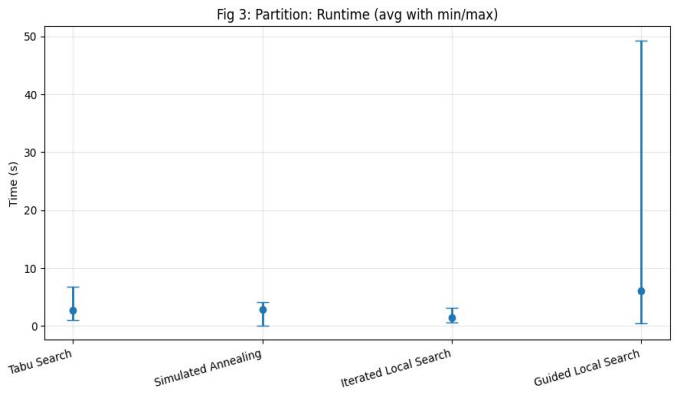
Detailed parameters for each algorithms

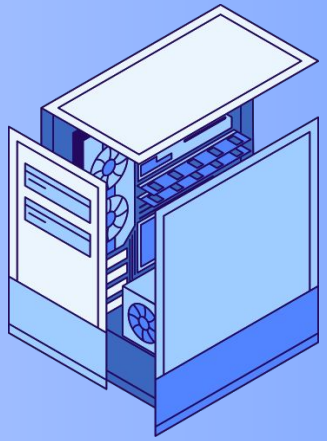
Algorithm	Parameters
TS	`max_iter=20000, tabu_tenure=10`
SA	`T0=1.0, cooling rate=0.995, iters=20000`
ILS	`outer_iters=20000, k_perturb=4, ls_cap=10_000, accept_worse_prob=0.05, stagnation_limit=50`
GLS	`outer_iters=200, ls_cap=50_000, alpha=0.3`



Results







Compare with avg (Points in each graphs)

In order of better result

Objective : TS = ILS > GLS > SA

Runtime : ILS > TS > SA > GLS

In this case order of better result

Overall : ILS > TS > SA > GLS

Hyper Parameter Optimization

Hyper Parameter Optimization has been done to optimize the algorithms which were applied to Partition Problem using the library called optuna in python.

Optimized Parameters:

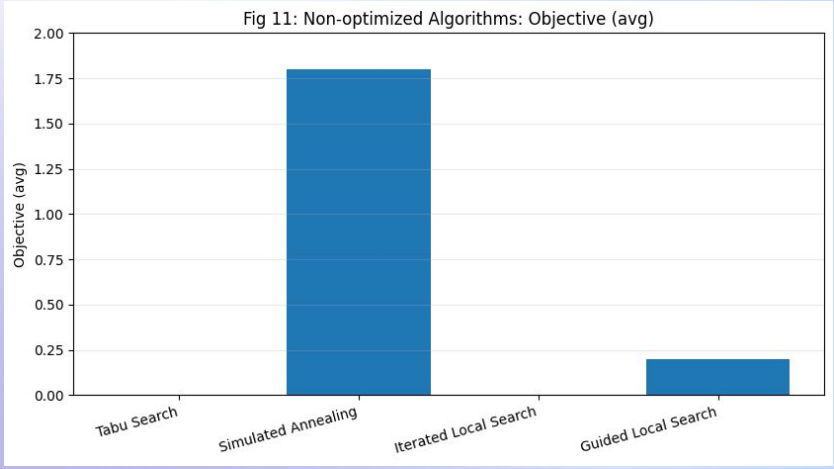
Tabu Search(TS): tabu tenure, no improvement limit(based on the maximum iterations), restart tries, seed(for randomness)

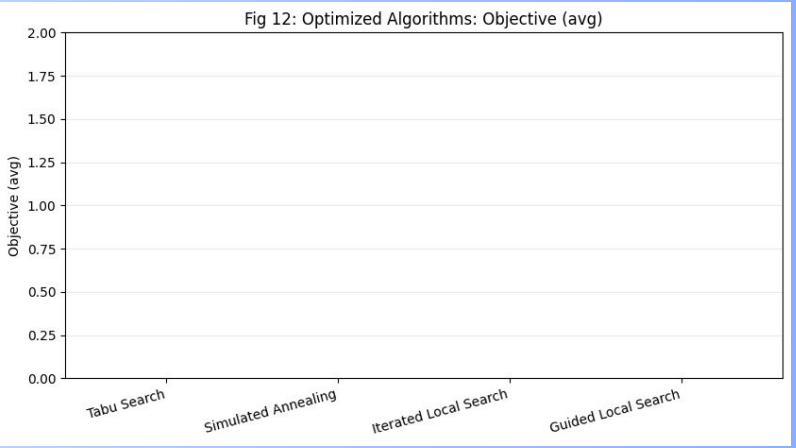
Simulated Annealing(SA): initial temperature, Cooling rate(alpha), iterations, seed.

Iterative Local Search(ILS): Number of iterations per local search, perturbation strength(k_perturb), probability for accepting worse solutions, stagnation limit, seed.

Guided Local Search(GLS): Outer iterations of the algorithm, iterations per local search, penalty weight(alpha), seed.

Improvement of the average solution quality of algorithms after HPO

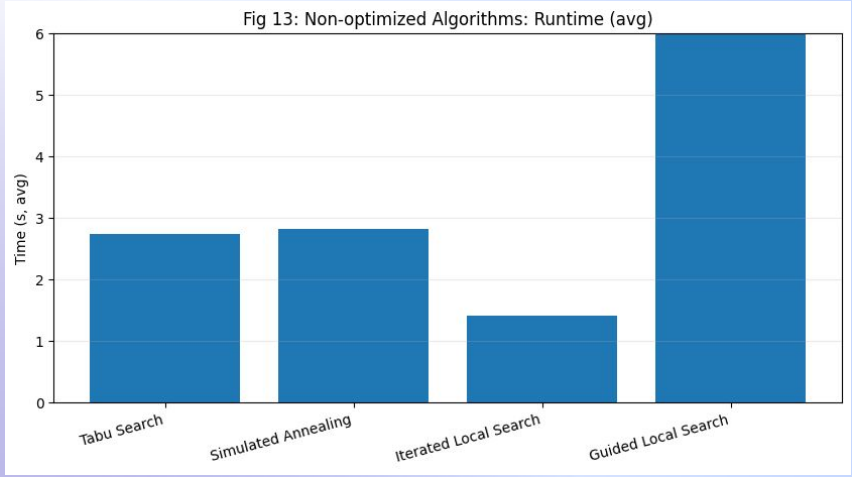


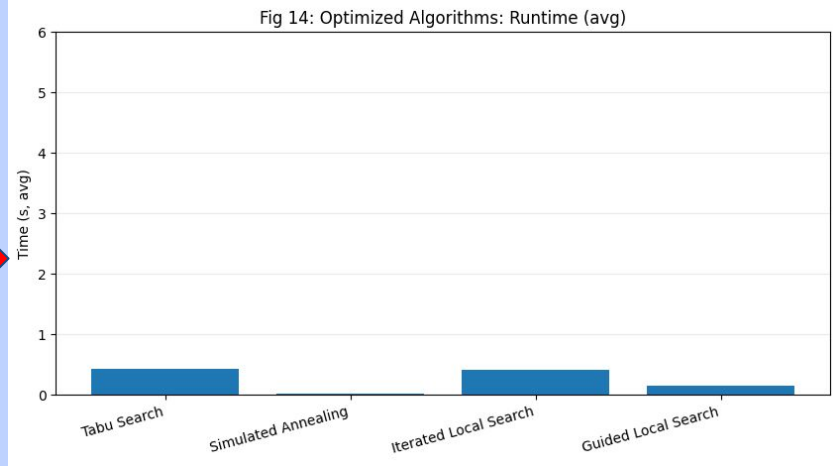


TS = ILS > GLS > SA

TS=ILS=GLS=SA

Improvement of Average Runtime Efficiency of Algorithms after HPO.

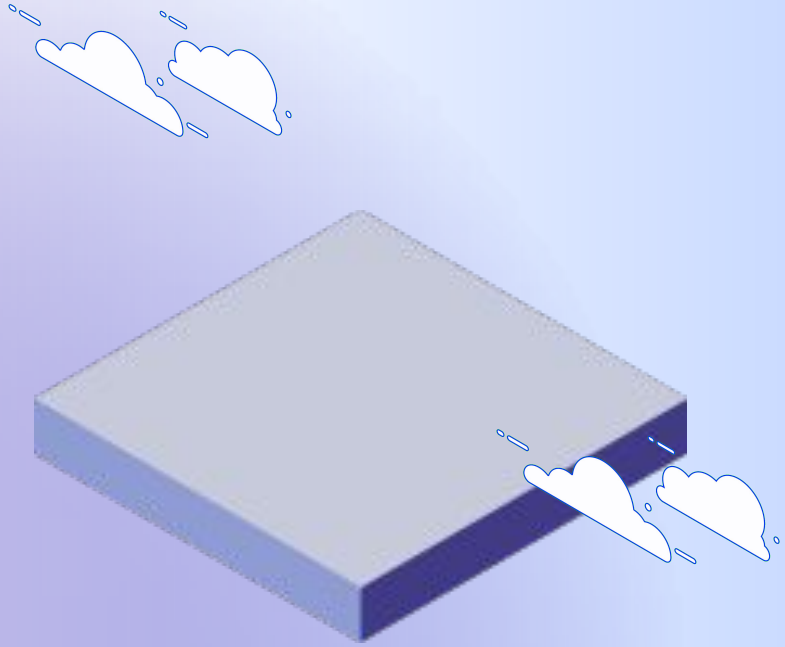




ILS > TS > SA > GLS

SA > GLS > TS > ILS

Cross-Problem Comparison



Knapsack Problem (Problem Size: 5 items, capacity 26):

- All algorithms achieved optimal value of 51
- SA fastest at 0.003s, GLS slowest at 0.020s
- Small problem size minimized performance differences.

Partition Problem (Problem Size: 10,000 integers):

- **Without optimization:**

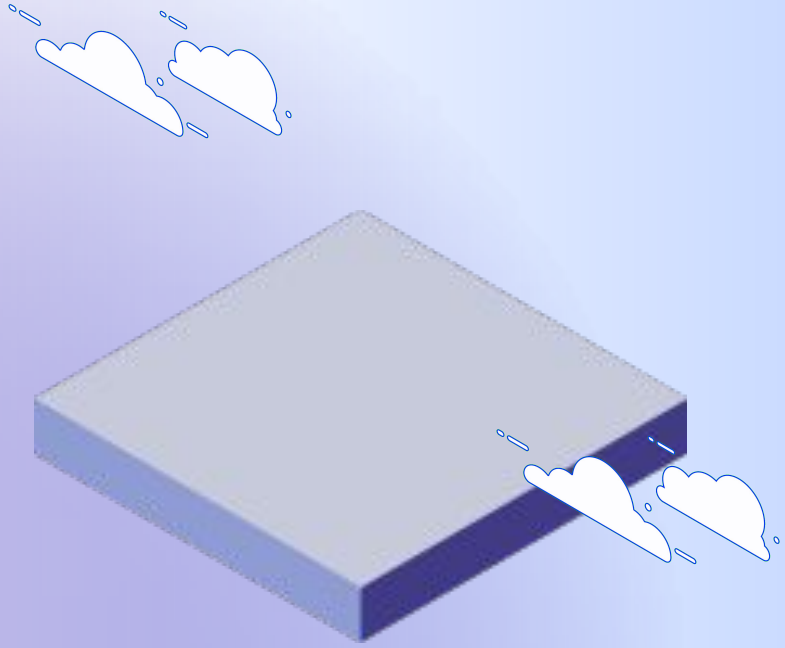
  ILS and TS achieved perfect partition (0), SA and GLS near-optimal.

- **With optimization:**

  All algorithms achieved perfect partition.

- **Runtime improvements:**

  SA improved 99.9% (2.39s → 0.002s), GLS improved 96% (5.24s → 0.21s)



Cross-Problem Comparison

Best Performance

- **Knapsack problem:**

  SA (dominating in the aspect of average runtime)

- **Partition Problem:**

  ILS (when considering both the quality of the solution and the runtime efficiency balance)

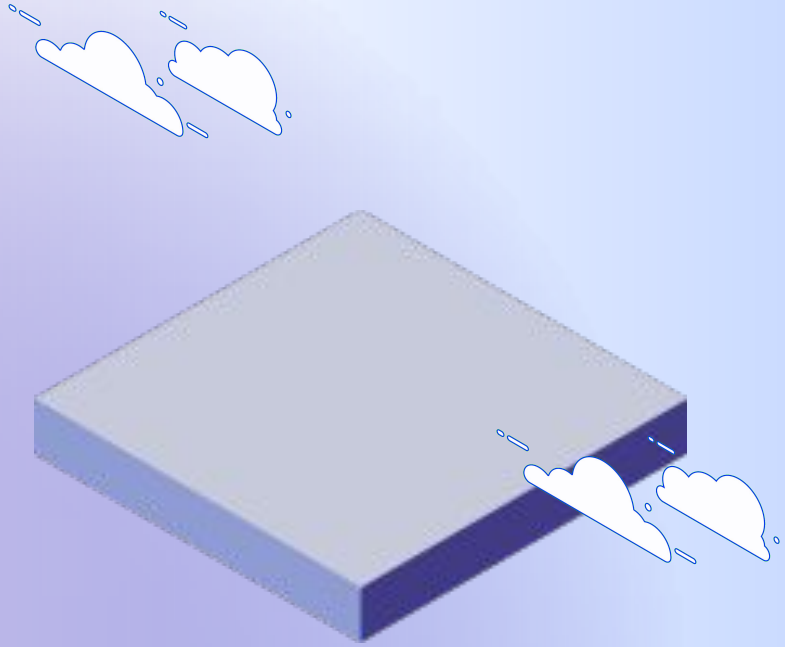
- **Optimized Partition Problem:**

  SA (by dominating the average runtime)

Improvements after Optimization of Algorithms in Partition Problem:

- Average Objective Value: SA.
- Average Runtime: GLS (best runtime is reached by SA).

Key Findings



- SA is the best performing algorithm for Knapsack problem with a smaller number of inputs.
- ILS was the best performing algorithm(considering both Runtime average and Objective value average balance) in Partition problem without optimization.
- SA becomes the best performing algorithm for Partition problem after Hyper Parameter Optimization.
- Therefore, by considering optimized version of Partition problem and knapsack problem it has been identified that SA has dominated with respected to runtime efficiency.
- Hyperparameter optimization can yield 10-100x performance gains in Partition Problem.
- No single algorithm dominates all scenarios.

THANK YOU!

Feel Free to ask any questions…..